Network Packet Analysis & Anomaly Detection (ICMP Analysis)

1. Objective

The objective of this project is to analyse network traffic using Wireshark to capture, filter, and study ICMP packets.
The project focuses on identifying high-frequency ICMP traffic (ping behaviour) which may indicate network scanning, misconfiguration, or potential denial-of-service patterns.

2. Tools & Environment Used

- **Operating System:** Kali Linux (VirtualBox)
- **Packet Analyzer:** Wireshark
- **Network Interface:** eth0
- **Target Domain:** example.org / example.com
- **Protocols Analysed:** ICMP, TCP, DNS
- **Capture File Format:** .pcap

3. Network Setup

- Virtual Machine connected using Bridged Adaptor networking
- Traffic captured on eth0
- Internet access verified using ping and browser access

4. Packet Capture Process

Step 1: Starting Packet Capture

- Wireshark was launched
- Interface **eth0** was selected
- Live Capture was started

Step 2: Generating ICMP Traffic

To generate ICMP packets, the following command was used:

“sudo ping -4 example.com -i 0.01”

- -4 forces IPv4
- -i 0.01 sends packets at high frequency (interval below 0.2s requires root/sudo)

  This simulates **intensive ICMP traffic**.

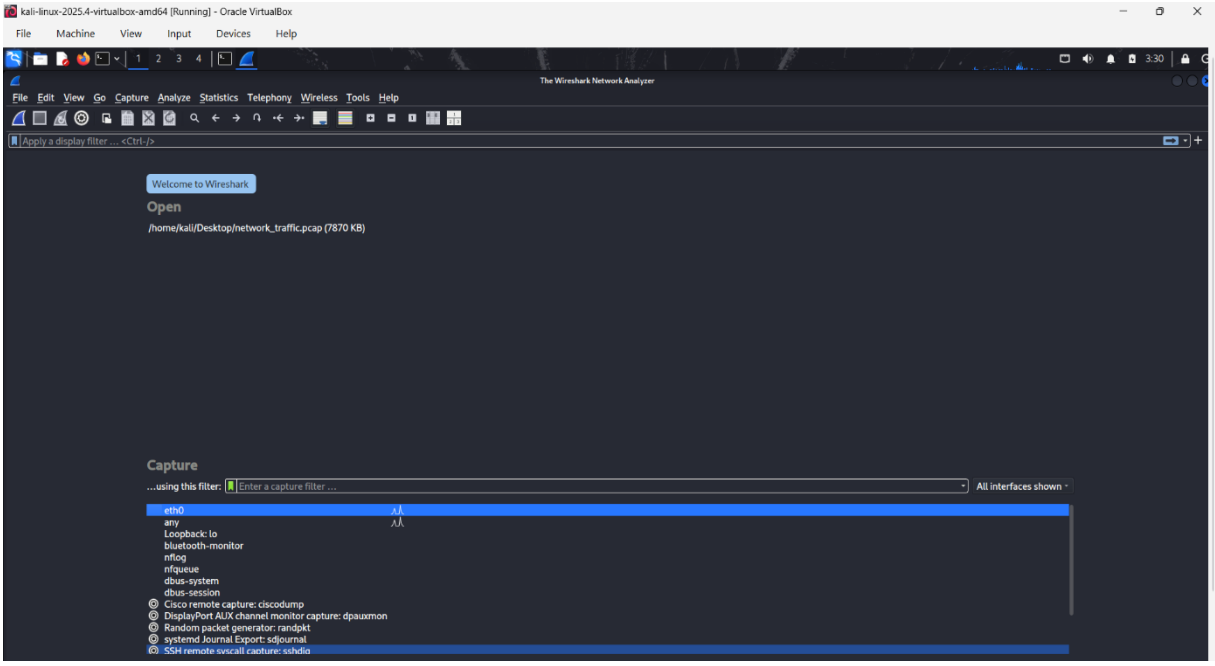

Figure 1: Wireshark interface

5. ICMP Packet Filtering

Step 3: Applying ICMP Display Filter

Wireshark display filter used: **"icmp"**

This isolates only ICMP packets from the capture.

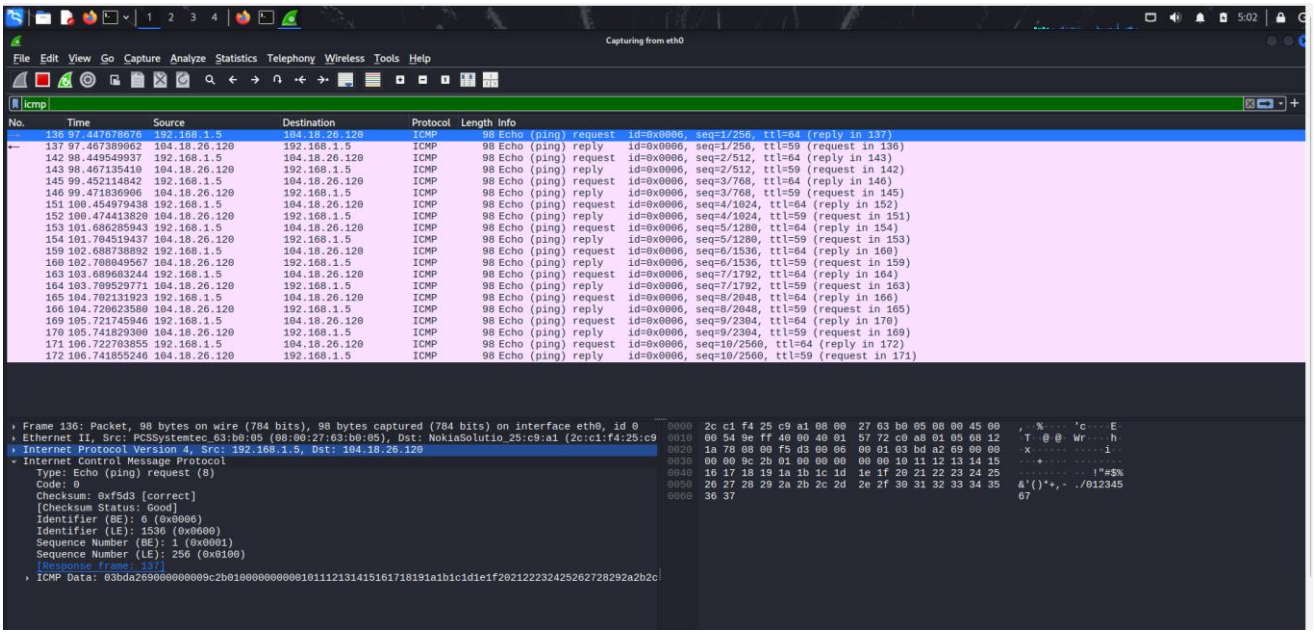

Figure 2: icmp Packet Capture

6. ICMP Packet Analysis

Step 4: Echo Request & Reply Identification

Captured packets clearly show:

- Echo (ping) request
- Echo (ping) reply
- Matching request–response frames

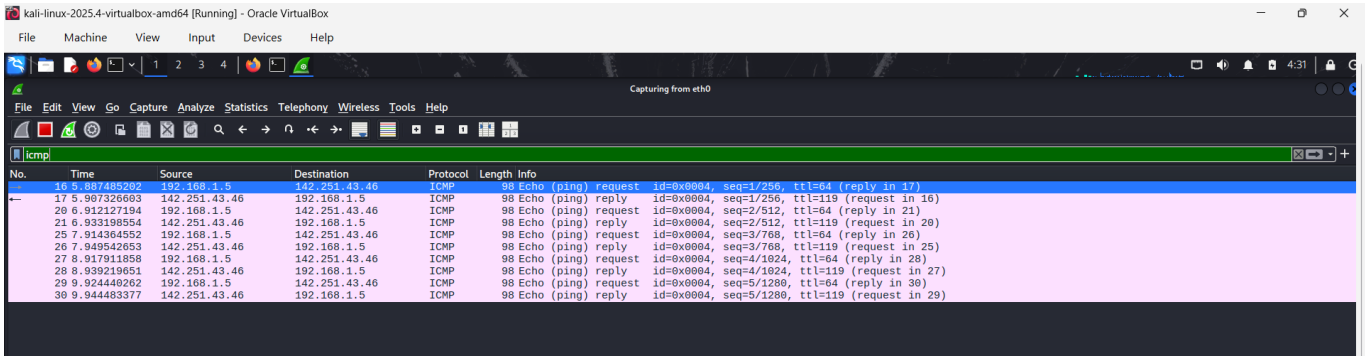

Figure 3: icmp echo request reply list

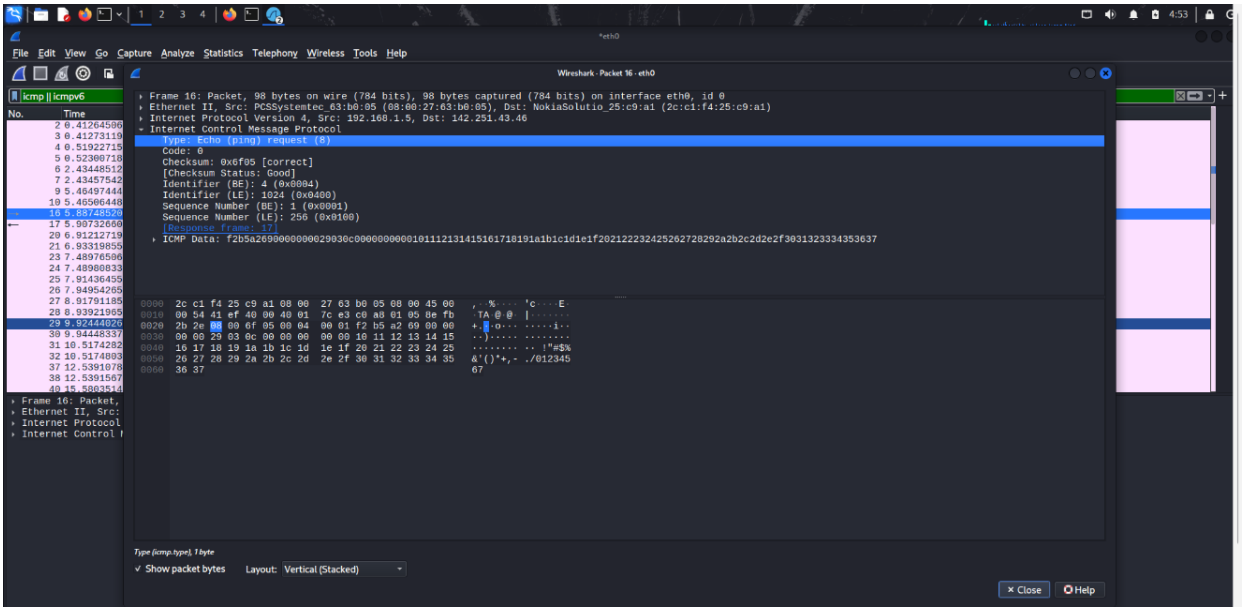

Figure 4: icmp Packet Details

Step 5: Packet-Level Details

From packet inspection:

- ICMP Type: 8 (Request), 0 (Reply)
- Identifier and sequence numbers increment correctly
- Response time visible per packet

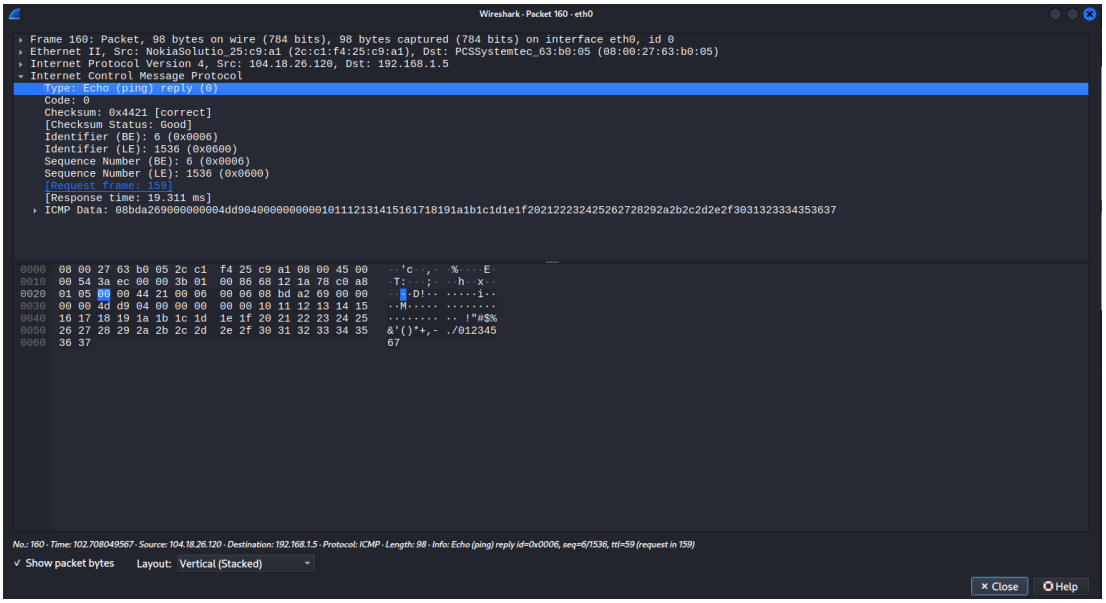

Figure 5: icmp echo request reply

7. High-Frequency ICMP Detection

Step 6: Detecting ICMP Flood Behaviour

Due to the high ping rate:

- Large number of ICMP packets captured
- Traffic density visibly increased
- Continuous echo requests observed

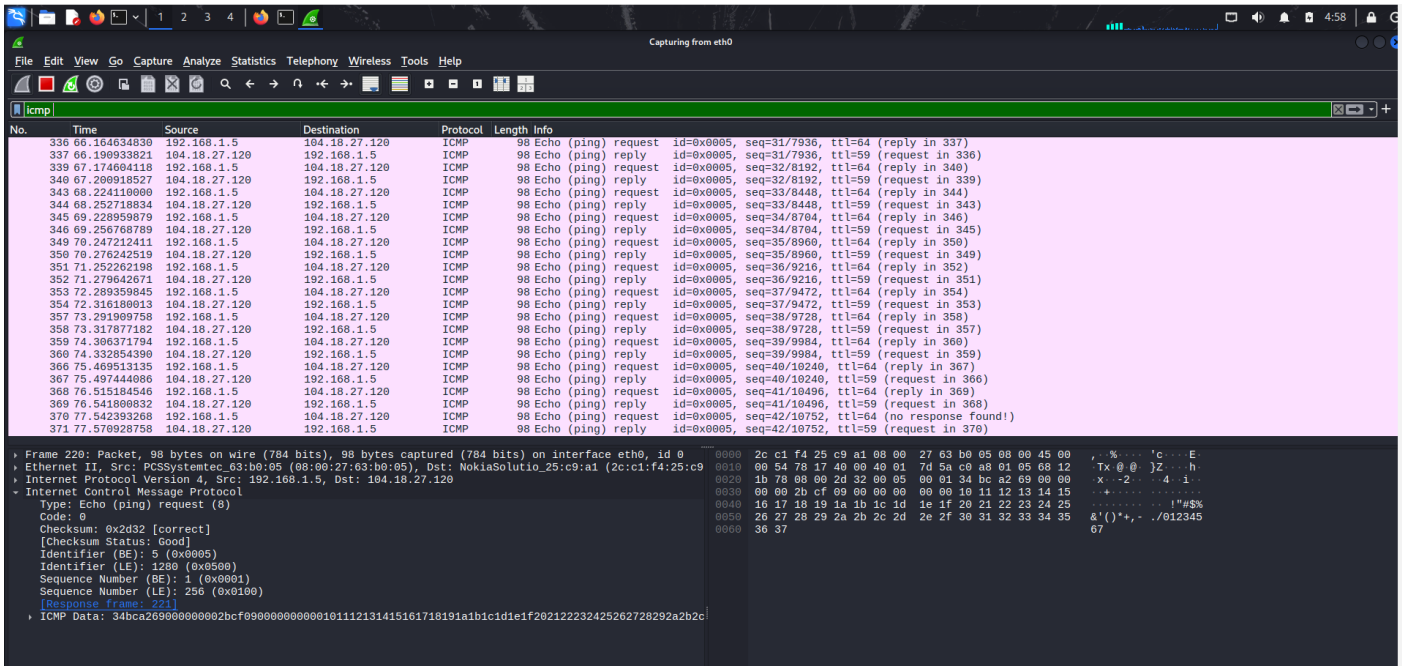

Figure 6 : icmp high frequency ping

8. ICMP Conversations Analysis

Step 7: Conversation Statistics

Wireshark path:

Statistics → Conversations → IPv4

Findings:

- One dominant conversation
- High packet count between local IP and destination
- Symmetrical packet flow

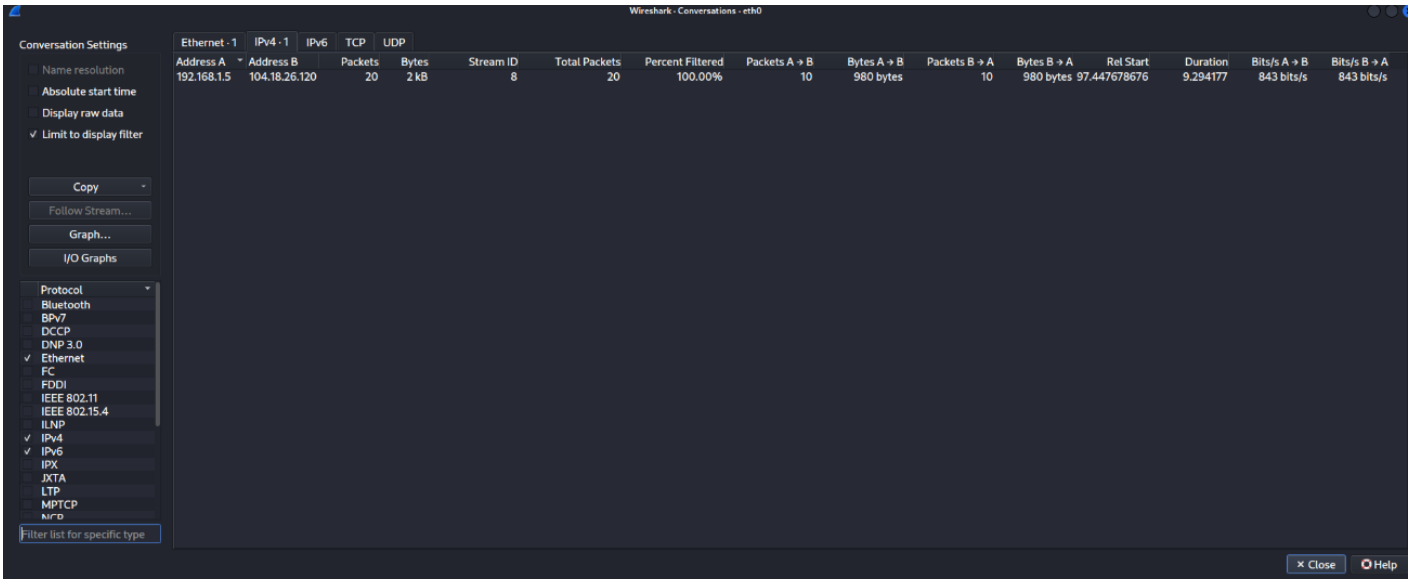

Figure 7: icmp Conversations Sorted

9. Traffic Visualization (Alert Evidence)

In this implementation, anomalies are identified through visual traffic spikes in the I/O Graph. Note: This is a manual visual detection approach, not an automated alert system. Automated alerting (e.g., using Python + Scapy) is listed as a future enhancement.

Step 8: I/O Graph Analysis

Path:

Statistics → I/O Graphs

Settings used:

- Interval: **1 sec**
- Y-Axis: **Packets**

- Display: ICMP Packets

Observation:

- Sudden spike in packets/sec
- Sustained high ICMP traffic
- Clear visual indicator of abnormal activity

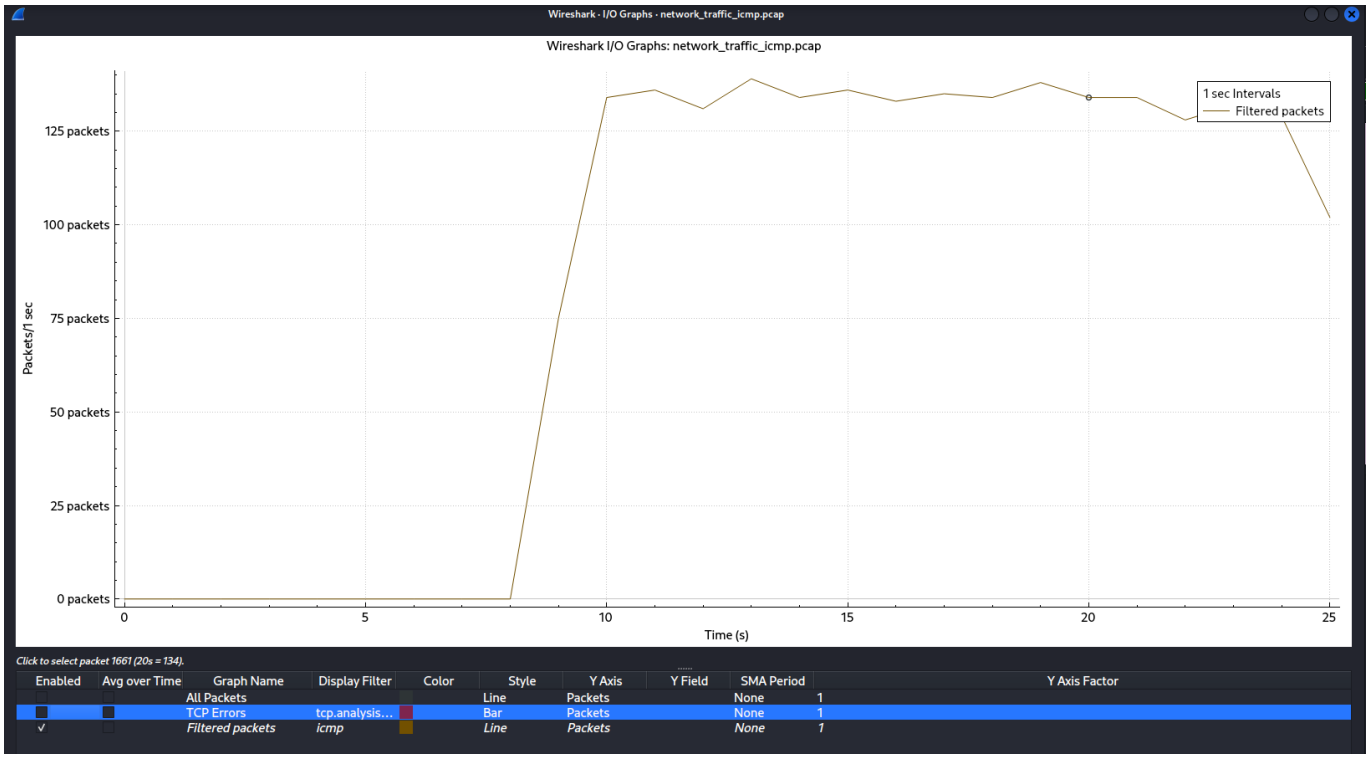

Figure 8: icmp I/O Graph Packets

10. Additional Protocol Observations

Although the focus was ICMP, other protocols were observed:

TCP SYN Packets

- Filter used:

tcp.flags.syn == 1

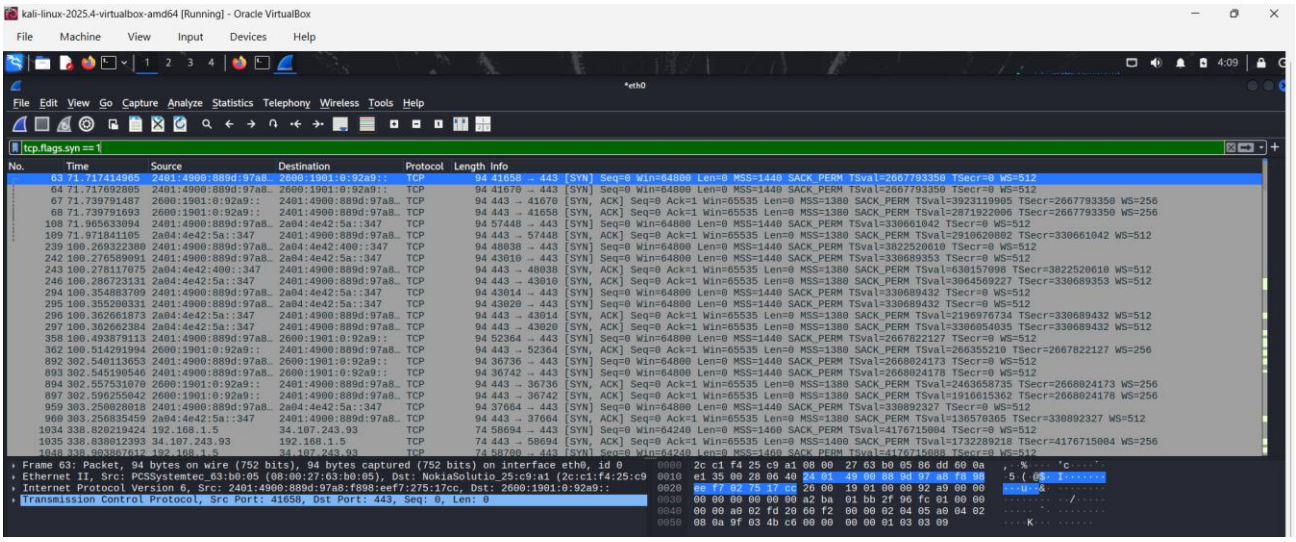

Figure 9: TCP SYN Scan Packets

11. Packet Capture File

- Capture saved as:

network_traffic_icmp.pcap

This file contains all raw packet data for verification and re-analysis.

12. Conclusion

This project successfully demonstrates:

- Live packet capture
- ICMP filtering and inspection
- Detection of high-frequency ICMP traffic
- Use of graphs for traffic anomaly visualization

Such ICMP behaviour can indicate:

- Network diagnostics misuse
- Automated scanning
- Early signs of DoS-style activity

Wireshark proved effective in identifying and visualizing these patterns.

13. Future Enhancements

- Automate alerts using scripts (Python + Scapy)
- Threshold-based ICMP detection
- Integration with IDS tools
- Log analysis over time

14. Packet Capture Validity Statement

- The packet capture was performed on a bridged network interface, which gives the virtual machine a real network presence. The observed ICMP traffic was self-generated (from the local VM to example.com) for testing purposes and does not represent third-party or external traffic.
- The capture file (network_traffic_icmp.pcap) contains raw packet data and can be independently verified using Wireshark for authenticity and reproducibility.